

Notes: Reducing Overfitting in PyTorch Neural Network (Fashion MNIST)

Topic 1: Understanding the Problem – Overfitting

Explanation (In-depth):

We are working with the **Fashion MNIST** dataset. It contains 60,000 training images and 10,000 test images of clothes (t-shirts, shoes, jackets, etc.) – 10 different classes. This is a **multi-class classification** problem. We built a neural network using PyTorch. Initially, we trained on CPU using only 5,000 images (accuracy ~83%). Then we used GPU and trained on all 60,000 images; test accuracy improved to ~88.7%.

However, when we evaluated the same model on **training data**, accuracy was **98%**. On **test data** it was only 88.7%. This large gap (about 10%) is a classic sign of **overfitting**. The model has memorized the training data but does not generalize well to new, unseen data. Overfitting is bad because real-world performance will be poor.

Key takeaway: Overfitting = high training accuracy, significantly lower test accuracy.

Topic 2: Solutions to Overfitting

There are several ways to reduce overfitting. We will focus on three:

1. **Dropout** – randomly turns off neurons during training.
2. **Batch Normalization** – normalizes activations to stabilize training (has a side regularization effect).
3. **L2 Regularization (Weight Decay)** – adds a penalty term to the loss function to keep weights small.

We will apply all three to our existing neural network architecture:

- Input layer: 784 nodes (28×28 pixels)
 - Hidden layer 1: 128 neurons
 - Hidden layer 2: 64 neurons
 - Output layer: 10 neurons (one per class)
-

Topic 3: Dropout – Detailed Explanation

Explanation:

Dropout is a technique where during **each forward pass** (training only), we randomly “turn off” a fraction of neurons in a hidden layer. Turning off means removing all incoming and outgoing connections for those neurons. This forces the network to not rely too much on any single neuron. The network becomes simpler and more robust, reducing overfitting.

- **When to apply:** After the activation function (e.g., ReLU) on hidden layers. Not on output layer.

- **Parameter p:** Probability of **dropping** a neuron. Common values: 0.2, 0.3, 0.5. If $p=0.3$, 30% of neurons are turned off randomly.
- **Important:** Dropout is **only active during training**. During evaluation (testing), all neurons are used, but their outputs are scaled automatically by PyTorch.

Code Example (adding dropout to our model):

```
import torch.nn as nn

class FashionMNISTNN(nn.Module):
    def __init__(self):
        super(FashionMNISTNN, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 10)
        self.dropout = nn.Dropout(p=0.3)  # 30% dropout

    def forward(self, x):
        x = x.view(-1, 784)  # flatten
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)  # dropout after activation
        x = torch.relu(self.fc2(x))
        x = self.dropout(x)  # dropout also on second hidden layer
        x = self.fc3(x)
        return x
```

Output (conceptual):

During training, each forward pass uses a slightly different “thinned” network. During evaluation, dropout is automatically disabled – no output change needed.

Topic 4: Batch Normalization – Detailed Explanation

Explanation:

Batch Normalization (BatchNorm) addresses a problem called **internal covariate shift**. As weights update during training, the distribution of inputs to each layer keeps changing, making training unstable. BatchNorm normalizes the activations of a layer **for each mini-batch** so that they have a mean of 0 and variance of 1. Then it learns to scale and shift (using parameters γ and β) to allow the network to decide the optimal distribution.

- **Where to apply:** After the linear layer (`nn.Linear`) but **before** the activation function (e.g., ReLU). Usually applied to hidden layers, not the output layer.
- **Benefits:**
 - Stabilizes training (reduces internal covariate shift)
 - Allows higher learning rates (faster training)

- Provides a slight regularization effect (because each mini-batch uses different mean/variance, adding noise)
- **During evaluation:** BatchNorm uses running averages calculated during training (not the current batch statistics).

Code Example (adding BatchNorm to our model):

```
class FashionMNISTNN(nn.Module):
    def __init__(self):
        super(FashionMNISTNN, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.bn1 = nn.BatchNorm1d(128)    # 128 = number of neurons in
        # fc1 output
        self.fc2 = nn.Linear(128, 64)
        self.bn2 = nn.BatchNorm1d(64)    # 64 = neurons in fc2 output
        self.fc3 = nn.Linear(64, 10)
        self.dropout = nn.Dropout(p=0.3)

    def forward(self, x):
        x = x.view(-1, 784)
        x = self.fc1(x)
        x = self.bn1(x)                    # BatchNorm before activation
        x = torch.relu(x)
        x = self.dropout(x)

        x = self.fc2(x)
        x = self.bn2(x)                    # BatchNorm before activation
        x = torch.relu(x)
        x = self.dropout(x)

        x = self.fc3(x)
        return x
```

Output (conceptual):

During training, each mini-batch's activations are normalized. During evaluation, stored statistics are used. No visible output difference, but training becomes more stable and overfitting reduces.

Topic 5: L2 Regularization (Weight Decay) – Detailed Explanation

Explanation:

L2 regularization adds a penalty term to the loss function. The original loss (e.g., cross-entropy) is combined with the sum of squares of all weights:

$$\text{Total Loss} = \text{Original Loss} + \lambda * \text{sum}(\text{weight}^2)$$

Here, λ (**lambda**) is a hyperparameter that controls regularization strength. Adding this penalty discourages large weights. Large weights often cause overfitting because they make the model

too sensitive to specific features. By keeping weights small, the model distributes importance more evenly and generalizes better.

- **L1 vs L2:** L2 uses squared sum; L1 uses absolute sum. In deep learning, L2 is more common.
- **Weight Decay:** In PyTorch, L2 regularization is implemented via the `weight_decay` parameter in the optimizer. `weight_decay` is exactly the λ value.
- **Apply to weights only** (not biases) – PyTorch does this automatically.
- **Only during training** – no effect during evaluation.

Code Example (adding weight decay to optimizer):

```
# Assuming model, loss function, and optimizer are defined
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,
weight_decay=1e-4)
# weight_decay = 0.0001 ( $\lambda$  value)
```

Note: `weight_decay=1e-4` means $\lambda = 0.0001$. Typical values range from $1e-5$ to $1e-3$.

Topic 6: Putting It All Together – Training and Results

We modified our original code by:

- Adding `nn.Dropout(0.3)` after each hidden layer's activation.
- Adding `nn.BatchNorm1d(128)` and `nn.BatchNorm1d(64)` after linear layers but before activation.
- Setting `weight_decay=1e-4` in the Adam optimizer.

Then we retrained the model on the full 60,000 images for 100 epochs (same as before).

Evaluation Code (to check training vs test accuracy):

```
def evaluate(model, data_loader):
    model.eval() # VERY IMPORTANT: turns off dropout and uses
running stats for BatchNorm
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in data_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    accuracy = 100 * correct / total
    return accuracy

# Evaluate on test data
```

```
test_acc = evaluate(model, test_loader)
print(f"Test Accuracy: {test_acc:.2f}%")

# Evaluate on training data
train_acc = evaluate(model, train_loader)
print(f"Train Accuracy: {train_acc:.2f}%")
```

Output (after applying dropout + batch norm + weight decay):

```
Test Accuracy: 88.30%
Train Accuracy: 93.20%
```

Comparison before optimization:

- Before: Train Acc = 98%, Test Acc = 88.7% → Gap = 9.3%
- After: Train Acc = 93.2%, Test Acc = 88.3% → Gap = 4.9%

Conclusion: The gap reduced significantly (from ~10% to ~5%). Overfitting is reduced. The model now generalizes better to unseen data. Test accuracy remained almost the same (slight drop from 88.7% to 88.3% is acceptable because we trade a tiny bit of training performance for much better generalization).

Summary for Your Copy (Quick Recap)

Technique	Purpose	Code Snippet (PyTorch)
Dropout	Randomly turn off neurons during training	<code>nn.Dropout(p=0.3)</code>
BatchNorm1d	Normalize activations before activation	<code>nn.BatchNorm1d(num_features)</code>
L2 Regularization	Penalize large weights via weight decay	<code>optimizer = Adam(..., weight_decay=1e-4)</code>

Important: Always set `model.eval()` before evaluation to disable dropout and fix BatchNorm statistics. Use `model.train()` to re-enable them for training.

Final Note

These three techniques (dropout, batch norm, weight decay) are powerful and widely used to fight overfitting. You can experiment with different dropout probabilities (e.g., 0.2, 0.5) and weight decay values to further improve generalization. The next step would be hyperparameter tuning to boost test accuracy even more.